

CampusCare FastAPI API Contract

This document maps the current CampusCare application architecture into the API surface that should be exposed if the project is rebuilt using FastAPI.

The source of truth for the current app is the API gateway in `js/backend/gateway.js`, which exposes a single `window.API` facade composed from the following services:

- AuthService
- UserService
- TicketService
- AssignmentService
- StaffService
- MessageService
- FileService
- NoticeService
- SettingsService
- AuditService

1. Authentication APIs

These endpoints handle sign-in, session state, role checks, password reset, and logout.

Auth routes

Method	Route	Purpose
--------	-------	---------

GET	/api/auth/me	Return the current authenticated user profile/session info
GET	/api/auth/session	Check whether the user is logged in and get session state
POST	/api/auth/login	Log in a user (email/password, phone/password, Google-based flow, or admin login)
POST	/api/auth/logout	Log out the current user
POST	/api/auth/reset-password	Request/reset password for an email address
PUT	/api/auth/password	Update current user password
GET	/api/auth/admin-emails	Return the list of admin email addresses
GET	/api/auth/roles/{user_id}	Get role metadata for a user
POST	/api/auth/verify-role	Verify that a current user has a required role

Auth operations mapped from current gateway

- ADMIN_EMAILS
 - getCurrentUser()
 - onAuthChange(cb)
 - login(p)
 - _verifyRole(u, r)
 - resetPassword(e)
 - updateMyPassword(p)
 - logout()
-
-

2. User/Profile APIs

These endpoints manage the current user and their profile information.

Method	Route	Purpose
GET	/api/users/me	Get current user profile
PUT	/api/users/me	Save or update current user profile
GET	/api/users/me?force=true	Force refresh current profile from backend

User operations mapped from current gateway

- `getMyProfile(force)`
 - `saveMyProfile(p)`
 - `clear()` on logout cleanup
-

3. File Upload APIs

These endpoints handle upload of images/files and their storage metadata.

Method	Route	Purpose
POST	/api/files/upload	Upload a file or image to the configured folder
POST	/api/files/upload/{folder}	Upload a file into a given storage folder

File operations mapped from current gateway

- `uploadFile(file, folder, opts)`
-

4. Ticket APIs

This is the largest category and covers issue reporting, similar-ticket lookup, ticket follow/join flows, appointment, messaging, acknowledgment, resolution, reopening, and escalation.

Ticket listing and subscription

Method	Route	Purpose
GET	<code>/api/tickets</code>	List tickets with filters / scope
GET	<code>/api/tickets/confidential</code>	Get confidential tickets for the current user/admin
GET	<code>/api/tickets/stream</code>	Streaming or long-polling subscription for ticket updates
GET	<code>/api/tickets/confidential/stream</code>	Streaming confidential ticket updates

Ticket lifecycle

Method	Route	Purpose
POST	<code>/api/tickets</code>	Create a new ticket
GET	<code>/api/tickets/similar</code>	Search for similar open tickets
POST	<code>/api/tickets/{ticket_id}/join</code>	Join a ticket as a supporter / me-too participant
POST	<code>/api/tickets/{ticket_id}/follow</code>	Follow a ticket

DELETE	/api/tickets/ {ticket_id}/follow	Unfollow a ticket
POST	/api/tickets/ {ticket_id}/ appointment	Propose a repair/visit appointment window
PUT	/api/tickets/ {ticket_id}/ appointment	Respond to a proposed appointment
POST	/api/tickets/ {ticket_id}/messages	Add a ticket message
POST	/api/tickets/ {ticket_id}/ acknowledge	Technician acknowledges the ticket
POST	/api/tickets/ {ticket_id}/resolve	Mark a ticket as fixed/ resolved
POST	/api/tickets/ {ticket_id}/reopen	Reopen a ticket
POST	/api/tickets/ {ticket_id}/escalate	Escalate the ticket
POST	/api/tickets/ {ticket_id}/feedback	Submit feedback / rating

Ticket operations mapped from current gateway

- `subscribeToTickets(scope, cb, onError, onSync)`
- `subscribeToConfidentialTickets(uid, isAdmin, cb, onError, onSync)`
- `createTicket(p)`
- `findSimilarTickets(q)`
- `joinTicket(id)`
- `followTicket(id)`
- `unfollowTicket(id)`
- `proposeAppointment(id, window)`
- `respondToAppointment(id, decision)`

- `addTicketMessage(id, m)`
 - `acknowledgeTicket(id, techId, name)`
 - `resolveTicket(id, remarks, photo, materials)`
 - `reopenTicket(id, reason)`
 - `escalateTicket(id)`
 - `submitFeedback(id, rating)`
-

5. Assignment APIs

These endpoints handle auto-assignment and manual assignment of tickets to technicians.

Method	Route	Purpose
POST	<code>/api/assignments/tickets/{ticket_id}/assign</code>	Manually assign a ticket to a technician
POST	<code>/api/assignments/tickets/{ticket_id}/auto-assign</code>	Auto-assign a ticket
POST	<code>/api/assignments/waiting/auto-assign</code>	Auto-assign waiting tickets for a department
POST	<code>/api/assignments/waiting/auto-assign/{department}</code>	Auto-assign waiting tickets for a specific department
POST	<code>/api/assignments/waiting/auto-assign/{department}/{tech_id}</code>	Auto-assign waiting tickets using a specific technician as candidate

Assignment operations mapped from current gateway

- `assignTicket(id, techId, name)`
- `autoAssignTicket(id)`
- `autoAssignWaiting(dept)`

- `autoAssignWaitingAsTechnician(dept, techId)`
-

6. Staff and Campus Member APIs

These endpoints manage technicians, campus members, statuses, phone indexes, and profile photo requests.

Technician management

Method	Route	Purpose
GET	<code>/api/staff/technicians</code>	List technicians
POST	<code>/api/staff/technicians</code>	Create a technician record
PUT	<code>/api/staff/technicians/{tech_id}/status</code>	Update technician status
DELETE	<code>/api/staff/technicians/{tech_id}</code>	Delete a technician
POST	<code>/api/staff/technicians/{tech_id}/reset-password</code>	Reset technician password
GET	<code>/api/staff/technicians/temp-password-status</code>	Check whether temporary password mode is active

Campus members

Method	Route	Purpose
GET	<code>/api/staff/members</code>	List campus members

POST	/api/staff/members	Create a campus member
DELETE	/api/staff/members/{uid}	Delete a campus member
GET	/api/staff/members/phone/{phone}	Resolve staff email by phone
POST	/api/staff/members/rebuild-phone-index	Rebuild phone lookup index

Profile photo requests

Method	Route	Purpose
POST	/api/staff/profile-requests	Submit a profile photo request
GET	/api/staff/profile-requests	Subscribe to / list profile photo requests
PUT	/api/staff/profile-requests/{request_id}	Resolve a profile photo request

Staff operations mapped from current gateway

- `subscribeToTechnicians(scope, cb)`
- `createTechnician(p)`
- `subscribeToCampusMembers(cb)`
- `createCampusMember(p)`
- `deleteCampusMember(uid)`
- `resolveStaffEmailByPhone(phone)`
- `rebuildStaffPhoneIndex()`
- `updateTechnicianStatus(id, s)`
- `resetTechnicianPassword(id)`
- `isUsingTempPassword()`
- `deleteTechnician(id)`
- `submitProfilePhotoRequest(id, name, url)`
- `subscribeToProfileRequests(cb)`

- `resolveProfileRequest(id, ok)`
-

7. Message / Admin Messaging APIs

These endpoints handle admin messages, user-to-admin messaging, replies, and read-state tracking.

Method	Route	Purpose
POST	<code>/api/messages/admin</code>	Send an admin message
POST	<code>/api/messages/admin/conversation</code>	Start admin conversation with a recipient
POST	<code>/api/messages/admin/{message_id}/reply</code>	Reply to an admin message
GET	<code>/api/messages/admin</code>	List all admin messages / conversations
GET	<code>/api/messages/me/{user_id}</code>	List messages for the current user
PUT	<code>/api/messages/admin/{message_id}/read</code>	Mark an admin message as read
PUT	<code>/api/messages/thread/{thread_id}/read</code>	Mark a thread as read by a user

Message operations mapped from current gateway

- `sendAdminMessage(text, role, ticket)`
 - `adminStartConversation(r, text)`
 - `replyToAdminMessage(id, text, fromAdmin)`
 - `subscribeToAdminMessages(cb)`
 - `subscribeToMyMessages(uid, cb)`
 - `markAdminMessageRead(id)`
 - `markThreadReadByUser(id)`
-

8. Notice APIs

These endpoints manage public notices and their lifecycle.

Method	Route	Purpose
GET	/api/notices	List active notices
POST	/api/notices	Create a campus notice
PUT	/api/notices/{notice_id}/end	End / retire a notice

Notice operations mapped from current gateway

- `subscribeToNotices(cb)`
 - `postNotice(n)`
 - `endNotice(id)`
-

9. Settings APIs

These endpoints model app-level configuration and rules used by the admin panel.

Method	Route	Purpose
GET	/api/settings	Fetch current app settings
PUT	/api/settings	Save updated app settings

Settings operations mapped from current gateway

- `subscribeToSettings(cb)`

- `saveSettings(s)`
-

10. Audit Log APIs

These endpoints provide audit history for administrative review.

Method	Route	Purpose
GET	<code>/api/audit-log</code>	List audit log entries
GET	<code>/api/audit-log/stream</code>	Subscribe to live audit log updates

Audit operations mapped from current gateway

- `subscribeToAuditLog(cb)`
-

11. Diagnostic / Architecture Helper APIs

These are not user-facing endpoints, but they exist in the current gateway for architecture and debugging purposes.

Method	Route	Purpose
GET	<code>/api/diagnostics/events</code>	Return recent domain events
GET	<code>/api/diagnostics/services</code>	Return registered service names

Diagnostic operations mapped from current gateway

- `_events()`

- `_services`
-

12. Recommended FastAPI Router Layout

A clean FastAPI structure for this project would be:

```
app/
├── main.py
├── api/
│   ├── auth.py
│   ├── users.py
│   ├── files.py
│   ├── tickets.py
│   ├── assignments.py
│   ├── staff.py
│   ├── messages.py
│   ├── notices.py
│   ├── settings.py
│   ├── audit.py
│   └── diagnostics.py
├── core/
│   ├── config.py
│   ├── security.py
│   ├── firebase.py
│   └── deps.py
└── services/
    ├── auth_service.py
    ├── user_service.py
    ├── ticket_service.py
    ├── assignment_service.py
    ├── staff_service.py
    ├── message_service.py
    ├── file_service.py
    ├── notice_service.py
    ├── settings_service.py
    └── audit_service.py
```

```
└─ models/
    ├── user.py
    ├── ticket.py
    ├── staff.py
    ├── message.py
    ├── notice.py
    └── settings.py
```

13. Notes on the API design

The current application exposes a single facade-level API and internally splits concerns across independent service modules. In a FastAPI migration, you should keep the same logical boundaries, but convert them into a standard REST-style API grouped by domain.

The minimum API surface to preserve feature parity with the current app is:

1. Auth
2. Users
3. Files
4. Tickets
5. Assignment
6. Staff
7. Messages
8. Notices
9. Settings
10. Audit log

This is the full backend contract needed to preserve the current CampusCare feature set.